



Evidencias de pruebas

Alumno: Diego Javier Martínez Sánchez

Matrícula: A00845422

Materia: Estructura de datos y algoritmos fundamentales

Grupo: 607

Profesor: David Alonso Cantú

Fecha de entrega: 22 de septiembre de 2026

1. Plan de pruebas

El objetivo fue verificar la lectura de ambos archivos, la corrección del ordenamiento, la búsqueda por rango, el manejo de timestamps duplicados, la validación de entradas y el comportamiento observado de cada algoritmo en los dos conjuntos de datos.

Área	Validación	Criterio de aceptación
Lectura	Carga de ambos archivos	Se leen 6,818 registros
Ordenamiento	Siete algoritmos en ambos archivos	Los registros quedan en orden cronológico
Salida	output607.txt	Conserva el formato y el orden
Rango	Límites binarios	Resultados inclusivos sin duplicar registros
Duplicados	Límite exacto repetido	Se recuperan ambos registros
Errores	Opciones y fechas inválidas	Mensaje claro y nueva oportunidad
Desempeño	14 combinaciones	Tiempo guardado por corrida

Metodología

En cada corrida se cargó una copia nueva del archivo, se eligió un algoritmo, se registró una predicción, se midió únicamente la operación de ordenamiento y se verificó que la llave cronológica no disminuyera. Posteriormente se realizó la búsqueda por rango y se generaron los archivos de salida.

2. Casos de prueba funcionales

Caso	Entrada o acción	Resultado esperado	Estado
Archivo 1	log607-1.txt	6,818 registros	Aprobado
Archivo 2	log607-2.txt	6,818 registros	Aprobado
Opción inválida	17 en menú 1-2	Rechazo y nueva solicitud	Aprobado
Formato inválido	Octubre 02 2024 10:00:00	Mes rechazado	Aprobado
Día inválido	Feb 30 2025 12:00:00	Día rechazado	Aprobado
Hora inválida	Oct 02 2024 25:10:00	Hora rechazada	Aprobado
Rango invertido	Inicio posterior al fin	Se solicita nuevamente	Aprobado
Rango vacío	Timestamp inexistente	0 resultados	Aprobado
Duplicados	Oct 02 2024 23:04:24	2 resultados	Aprobado

Política para timestamps duplicados

La búsqueda usa un límite inferior que encuentra la primera llave mayor o igual al inicio y un límite superior que encuentra la primera llave estrictamente mayor al final. El subrango es inclusivo y conserva todos los registros repetidos que coincidan con los límites. Cada límite se obtiene en $O(\log n)$.

3. Resultados de tiempos

Algoritmo	Desordenado (μs)	Casi ordenado (μs)	Mejora
Swap Sort	7,261,659	173,415	41.9x
Bubble Sort	7,693,272	13,122	586.3x
Selection Sort	131,172	113,377	1.16x
Insertion Sort	6,881,336	10,270	670.0x
Merge Sort	68,011	66,309	1.03x
Quick Sort	17,658	7,132	2.48x
Shell Sort	49,544	35,032	1.41x

Interpretación

Quick Sort fue el algoritmo más rápido en ambos archivos. Insertion Sort mostró la mayor mejora relativa al pasar de aproximadamente 6.88 segundos a 10.27 milisegundos en el archivo casi ordenado. Bubble Sort también mejoró de forma marcada gracias a la parada anticipada. Selection Sort cambió poco porque sigue buscando el mínimo restante sin importar el orden inicial, y Merge Sort fue estable por su complejidad $O(n \log n)$.

4. Análisis comparativo

Algoritmo	Comportamiento observado	Interpretación
Swap	Mejóro en el casi ordenado	Mantiene $O(n^2)$ comparaciones, pero hace menos intercambios
Bubble	Cambio muy grande	La parada anticipada detecta pronto que ya no hay inversiones
Selection	Tiempos similares	Busca el mínimo restante en cada iteración
Insertion	Cambio más pronunciado	Con pocas inversiones realiza pocos desplazamientos
Merge	Muy estable	$O(n \log n)$ en mejor y peor caso
Quick	Más rápido	El pivote central funcionó bien en estos datos
Shell	Rápido con mejora moderada	Los saltos reducen movimientos antes de la inserción final

Conclusión

Los resultados confirman que el orden inicial cambia considerablemente el desempeño de los algoritmos adaptativos. Para los datos medidos, Quick Sort fue la opción más rápida en general, mientras que Insertion Sort fue especialmente conveniente cuando el archivo ya estaba casi ordenado. Merge Sort ofrece un comportamiento consistente cuando se prefiere un límite de complejidad $O(n \log n)$.

5. Evidencia visual

Captura 1. Fecha inválida

```
BUSQUEDA POR RANGO
=====

Ingresa la fecha y hora inicial.
Formato: Mon DD YYYY HH:MM:SS
Ejemplo: Oct 02 2024 23:04:24
> Feb 30 2025 12:00:00
Fecha u hora invalida: Dia invalido para el mes indicado: 30
```

Captura 2. Rango vacío

```
Registros encontrados: 0
No existen registros dentro del rango indicado.
```

Captura 3. Timestamps duplicados

```
Formato: Mon DD YYYY HH:MM:SS  
Ejemplo: Oct 02 2024 23:04:24  
> Oct 02 2024 23:04:24  
  
Ingresa la fecha y hora final.  
Formato: Mon DD YYYY HH:MM:SS  
Ejemplo: Oct 02 2024 23:04:24  
> Oct 02 2024 23:04:24  
  
Registros encontrados: 2
```

Captura 4. Rango invertido

```
La fecha inicial no puede ser posterior a la fecha final.  
Ingresa nuevamente el rango.
```

6. Recomendaciones

Para este volumen de datos, Quick Sort fue la mejor elección empírica. Merge Sort es una alternativa robusta cuando se busca consistencia teórica. Insertion Sort y Bubble Sort optimizado son adecuados cuando hay evidencia de que el archivo llega casi ordenado. Selection y Swap Sort no se recomiendan para grandes archivos desordenados debido a su costo cuadrático.

Los tiempos corresponden a una sola corrida por combinación en el mismo equipo. En un análisis experimental más amplio convendría hacer múltiples repeticiones y reportar la mediana.